Java Programming 3

Week 7 - JDBC - JdbcClient - Profiles

KdG

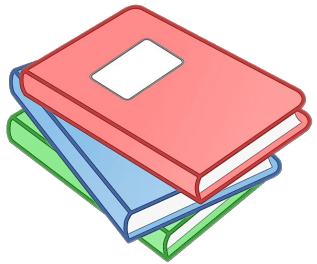
Agenda this week

JDBC
Implementing the repository
JdbcClient
Spring profiles

KdG

Tutorials

- JDBC: http://tutorials.jenkov.com/jdbc/index.html
- JDBC and Spring: https://www.baeldung.com/spring-6-jdbcclient-api
- Spring JDBC Reference: https://docs.spring.io/spring-framework/reference/data-access/jdbc.html
- Spring Profiles: https://www.baeldung.com/spring-profiles
- JdbcClient: https://www.baeldung.com/spring-6-jdbcclient-api



KdG

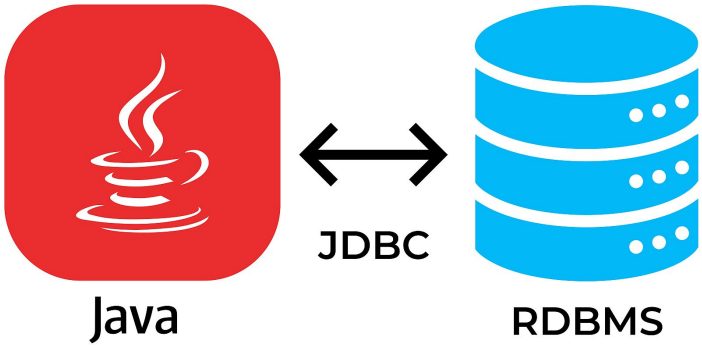
Agenda this week

JDBC
Implementing the repository
JdbcClient
Spring profiles

KdG

JDBC

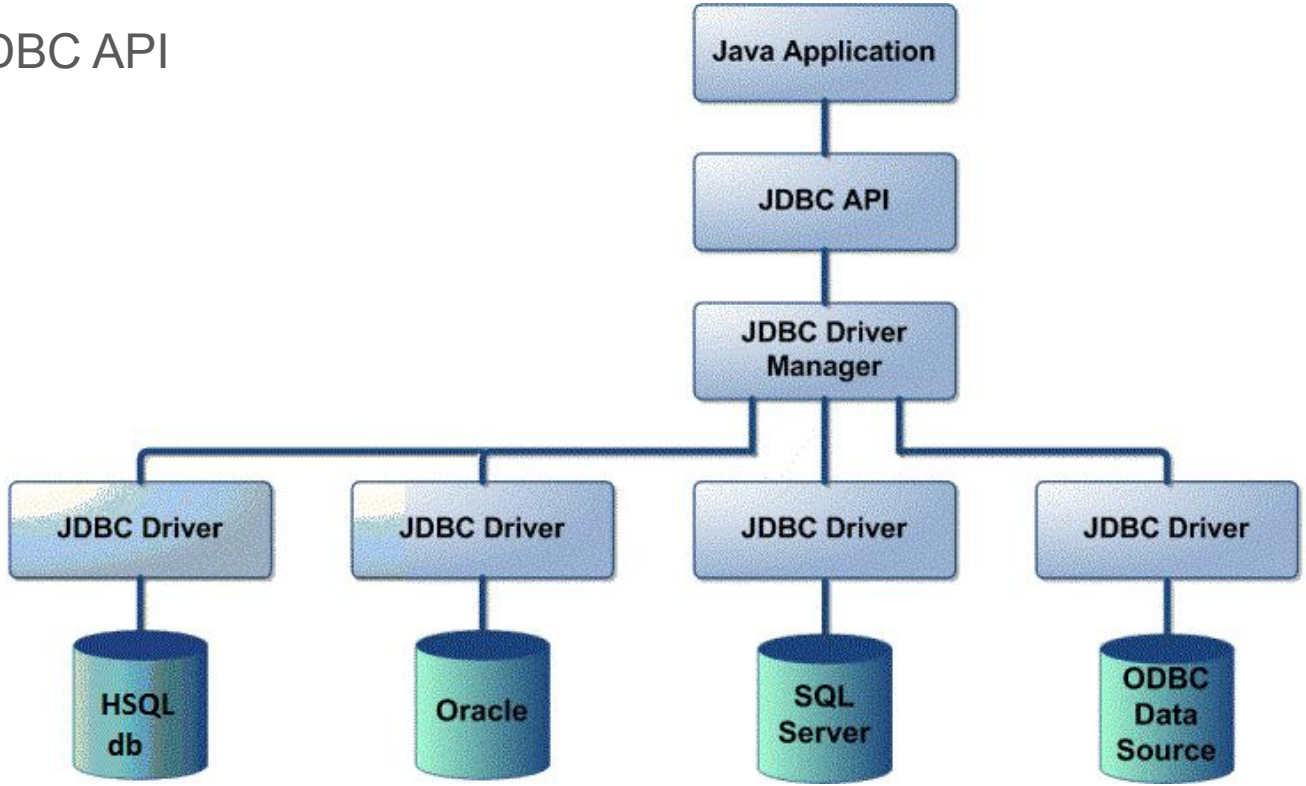
- API that allows a Java program to communicate with a relational database
- Independent of the vendor
- Access using SQL Queries
- Part of standard Java (java.sql.*)



KdG

JDBC Driver

- Implements the JDBC API
- Vendor specific



KdG

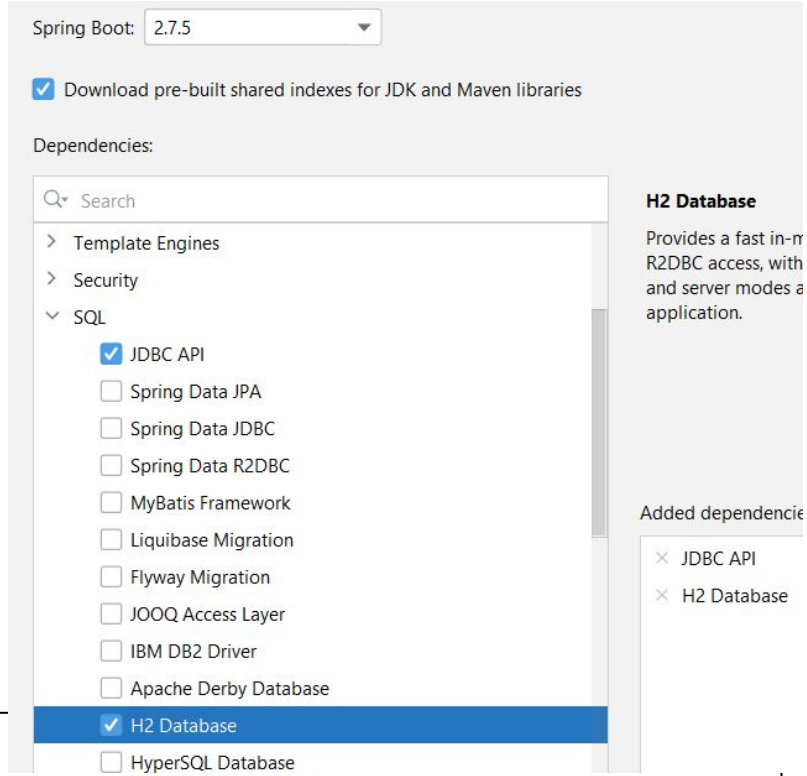
H2 Database

- We will use H2 DB in the examples
  - Lightweight, small, fast, in memory
  - Perfect for development and testing
  - Not suitable for production: you will use PostgreSQL instead
- We will have to add the driver to the gradle dependency!

KdG

Create new Spring project

- Add 2 dependencies
  - JDBC API
  - H2 Database



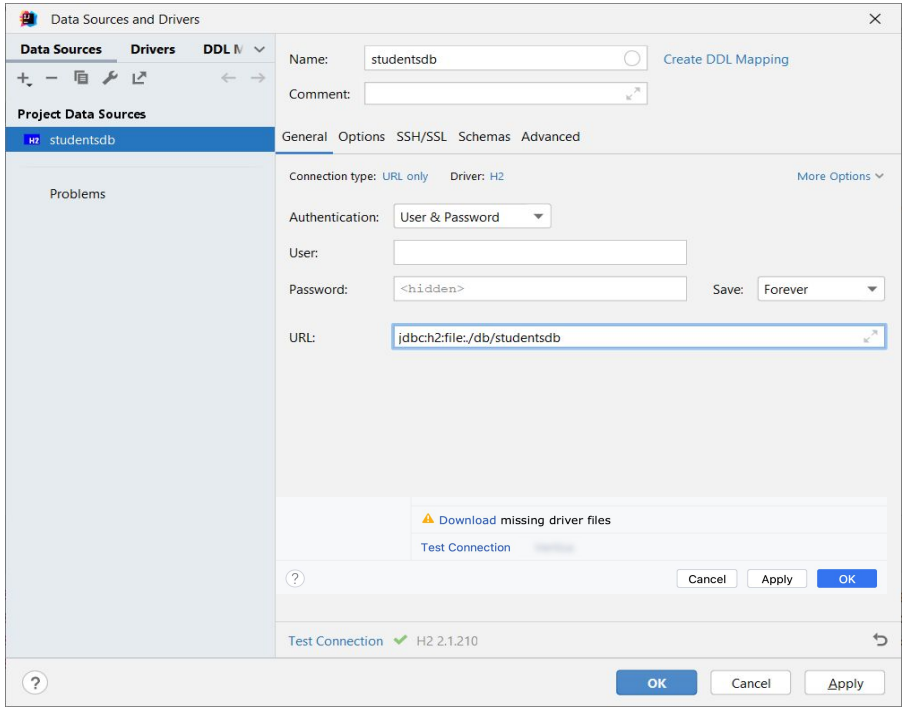

```
dependencies {
    //…
    implementation ("org.springframework.boot:spring-boot-starter-jdbc")
    runtimeOnly ("com.h2database:h2")
}
```

Create DB first: IntelliJ Database View

- Connection Type: URL Only
- URL: **jdbc:h2:file:./db/studentsdb**

  → Creates DB in ./db folder

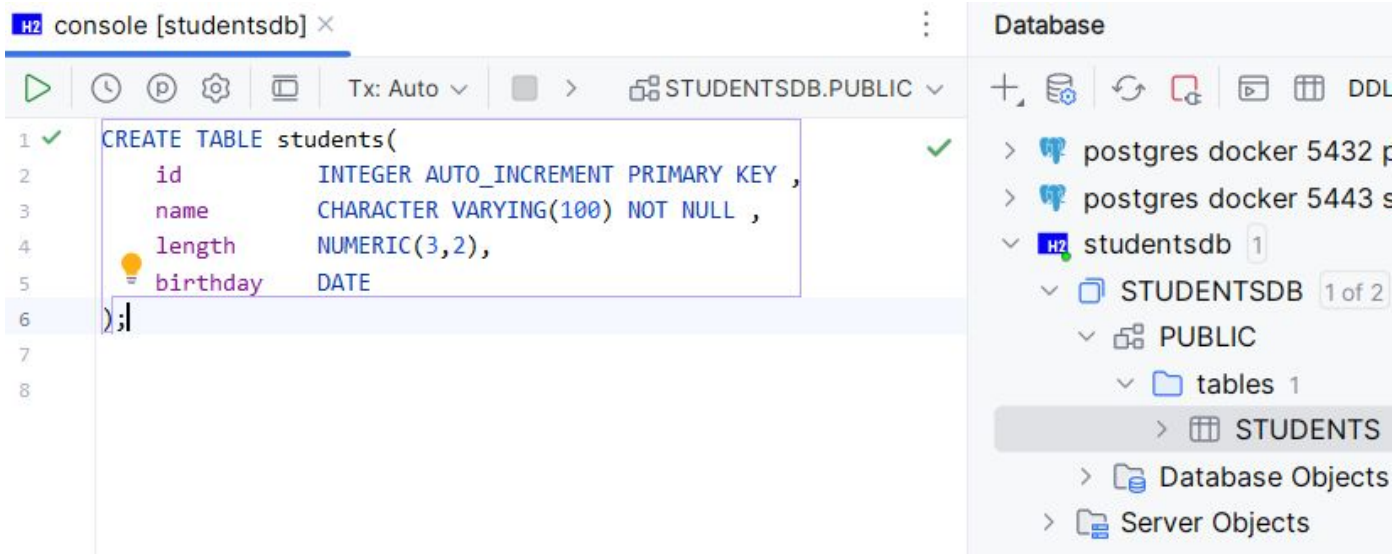
  - If intellij proposes to download missing driver files, do so.



KdG

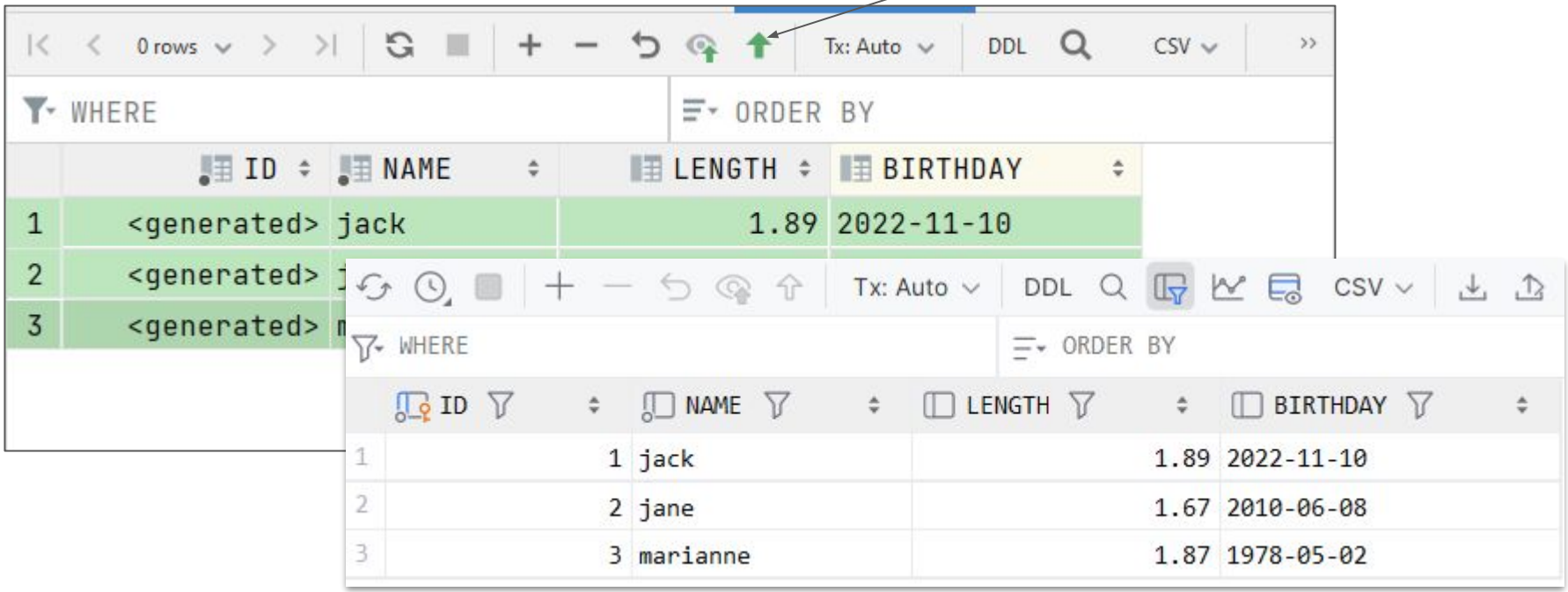
Create table STUDENT

- Use the DB View
- Create columns id (PK, auto_increment), name, LENGTH, BIRTHDAY



KdG

Add some records…



KdG

JDBC: basic steps

1. Add JDBC Driver
2. Create a `Connection` to the database
3. Create a `Statement` (or `PreparedStatement`)
4. Perform query
5. Process the result (`ResultSet`)
6. Close `ResultSet` - `Statement` - `Connection`

Slides code

KdG

Connection

- Create a connection to the Database via **`DriverManager.getConnection(..)`**
  - You can provide username and password as extra parameters

```
@Component
public class JdbcQuery implements CommandLineRunner {
 @Override
 public void run(String... args) throws Exception {
  Connection connection = DriverManager.getConnection("jdbc:h2:file:./db/studentsdb");
  Statement statement = connection.createStatement();
  ResultSet resultSet = statement.executeQuery("SELECT * FROM students");
  while (resultSet.next()) {
   System.out.printf("%d %s\n",resultSet.getInt("id"), resultSet.getString("name"));
  }
  resultSet.close();
  statement.close();
  connection.close();
 }
}
```

```
1 jack
2 jane
3 marianne
```

```
A file can only be opened by one process, deactivate the
database in your IntelliJ tool window before running this.
(This problem does not occur with network databases)
```

KdG

Closing the connection

- Close the connection if it is no longer used!
  - Typically the DB can only serve a limited number of connections at the same time…
- Use try (*with resources)* to be sure → will automatically close connection
  - You can open multiple resources in one try (with resources)

```
public void run(String... args) throws Exception {
 try (Connection connection = DriverManager.getConnection("jdbc:h2:file:./db/studentsdb");
      Statement statement = connection.createStatement()) {
  try (ResultSet resultSet = statement.executeQuery("SELECT * FROM students")) {
   while (resultSet.next()) {
    System.out.printf("%d %s\n",resultSet.getInt("id"), resultSet.getString("name"));
   }
  }
 }
}
```

KdG

Statement

- You create a Statement to perform a query on the database
  - You can create multiple Statements on a connection
- Close the Statement after use (eg using try with resources)!

```
public void run(String... args) throws Exception {
 try (Connection connection = DriverManager.getConnection("jdbc:h2:file:./db/studentsdb");
      Statement statement = connection.createStatement()) {
  try (ResultSet resultSet = statement.executeQuery("SELECT * FROM students")) {
   while (resultSet.next()) {
    System.out.printf("%d %s\n",resultSet.getInt("id"), resultSet.getString("name"));
   }
  }
 }
}
```

KdG

ResultSet

- The executeQuery method returns a ResultSet. Via the ResultSet you can retrieve the rows returned by the query…
  - you can execute multiple queries **sequentially** on a statement
- Close the ResultSet after use: you can only have one ResultSet open on a statement.

```
public void run(String... args) throws Exception {
 try (Connection connection = DriverManager.getConnection("jdbc:h2:file:./db/studentsdb");
      Statement statement = connection.createStatement()) {
  try (ResultSet resultSet = statement.executeQuery("SELECT * FROM students")) {
   while (resultSet.next()) {
    System.out.printf("%d %s\n",resultSet.getInt("id"), resultSet.getString("name"));
   }
  }
 }
}
```

KdG

ResultSet

- The next() method moves the cursor to the next row
  - It returns false if there is none

```
public void run(String... args) throws Exception {
 try (Connection connection = DriverManager.getConnection("jdbc:h2:file:./db/studentsdb");
      Statement statement = connection.createStatement()) {
  try (ResultSet resultSet = statement.executeQuery("SELECT * FROM students")) {
   while (resultSet.next()) {
    System.out.printf("%d %s\n",resultSet.getInt("id"), resultSet.getString("name"));
   }
  }
 }
}
```

KdG

ResultSet

- On a row, you can retrieve values of different columns using the column name or the column index (starts from **1**)
  - Using name is preferred

```
try (Connection connection = DriverManager.getConnection("jdbc:h2:file:./db/studentsdb");
    Statement statement = connection.createStatement()) {
 try (ResultSet resultSet = statement.executeQuery("SELECT * FROM students")) {
  while (resultSet.next()) {
   System.out.printf("%d %s\n",resultSet.getInt("id"), resultSet.getString("name"));
   System.out.printf("%d %s\n",resultSet.getInt(1), resultSet.getString(2));
  }
 }
}
```

KdG

ResultSet get…: what SQL types?

- getString(...): for character based types (char, varchar, varchar2, …)
  - Works on any type (does a toString)
- getInt(...), getLong(...): integer types (smallint, int, …)
- getFloat(...), getDouble(...): decimal types (decimal, real, double precision, …)
- getBoolean(...): boolean types
- getDate(...): for dates
  - Returns java.sql.Date object → convert it to LocalDate for use in Java…
- more…

```
Date birthDay = resultSet.getDate("BIRTHDAY");
LocalDate localDate = birthDay.toLocalDate();
```

KdG

Insert, delete or update data?

```
try (Connection connection = DriverManager.getConnection("jdbc:h2:file:./db/studentsdb")) {
 try (Statement statement = connection.createStatement()) {
  int result = statement.executeUpdate("""
   INSERT
   INTO students(name, length, birthday)
   VALUES('AN',1.65,'1967-3-12')
   """);
  System.out.println(result  + " row(s) inserted");

  result = statement.executeUpdate("""
   DELETE
   FROM students
   WHERE name = 'AN'
   """);
  System.out.println(result + " row(s) deleted");
 }
}
```

- Use executeUpdate method for updates (inserts, deletes)
- Returns number of rows affected (1 in the above cases)

KdG

Exercise:

- Create new Spring application
  - Add JDBC and H2 dependencies
- Use the Database view in IntelliJ to:
  - Create table students: a user has an id, name, length and birthday
  - Add some data
- Write a small application (commandlinerunner) that shows all students sorted by birthday
- Perform some updates and deletes on the data
- What happens if you keep the connection open on the IntelliJ Database tool?

KdG

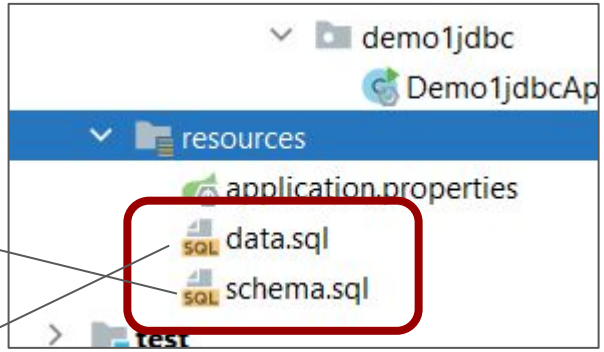
Creating the database tables and loading initial data?

- You can do this in database tool, or using Java code
- Or: you can add a `schema.sql` and `data.sql` to the resources folder
- Spring will run the .sql files once when the application starts...

```sql
DROP TABLE IF EXISTS persons; -- not needed for in memory DB

CREATE TABLE persons(
   id INTEGER AUTO_INCREMENT PRIMARY KEY,
   name CHARACTER VARYING(100) NOT NULL,
   firstname CHARACTER VARYING(50)  NOT NULL,
   remark CHARACTER VARYING(256)
);
```

```sql
INSERT INTO persons(name, firstname,remark)
VALUES ('JONES', 'JACK','of all trades'),
      ('POTTER', 'JACK','Lilly''s dad'),
      ('POTTER', 'MIA','Lilly''s mum'),
      ('REED', 'JACK','union');
```



Configure the datasource in Spring...

- Spring needs to know where to connect: you can add this info in the `application.properties`

```
spring.datasource.url=jdbc:h2:mem:personsdb
spring.datasource.username=sa
spring.datasource.password=
spring.sql.init.mode=always
```

We use a H2 **memory** database now: it will only exist in memory. Ideal for developing and testing…

If not using a memory database Spring does not run the schema.sql and data.sql, you need to add this last line...

Slides code: persons_datasource DataSourceRunner

KdG

Exercise:

- Create a small Spring application (no Spring MVC, just a commandlinerunner) that loads 10 students (name, length, birthday) into a H2 **memory** database at startup:
  - Configure in the application.properties
  - use schema.sql and data.sql to create the table and load the data
- The application asks for the name via the console
- The application shows all records that match the name
- Use this query: `"SELECT * FROM students WHERE name = '" + name + "'"`

```
Enter the name of the student:jack
Found following info for student jack:
id:1, name: "jack", length: 1.92, birthday: 10/11/2022
Enter the name of the student:jef
Found following info for student jef:
Enter the name of the student:matthijs
Found following info for student matthijs:
id:2, name: "matthijs", length: 1.67, birthday: 08/06/2010
Enter the name of the student:_
```

KdG

Exercise:

- Create a small Spring application (no Spring MVC, just a commandlinerunner) that loads 10 students (name, length, birthday) into a H2 memory database at startup:
  - Configure in the application.properties
  - use schema.sql and data.sql to create the table and load the data
- The application asks for the name via the console
- The application shows all records that match the name
- Use this query: `"SELECT * FROM students WHERE name = '" + name + "'"`

→ Try using `JACK' OR '1'='1` as name, what happens?

→ Example of SQL Injection!

KdG

PreparedStatement

The values are inserted at the ? in the PreparedStatement.
The prepared statement can execute the SAME statement multiple times with DIFFERENT parameters

- You can *prepare* a SQL statement:
  - The SQL will be precompiled in the DB → you can prepare it beforehand
  - You can change the parameters
  - It is a good protection against SQL injection…
  - You als get rid of the mix of double and single quotes in your statemen

```
try (PreparedStatement statement = connection.prepareStatement("INSERT INTO students(name, LENGTH, BIRTHDAY)
VALUES(?,?,?)")) {
    for (int i=0;i<10;i++) {
        statement.setString(1, "AN" + i);
        statement.setDouble(2, 1.78);
        statement.setDate(3, Date.valueOf(LocalDate.of(1987, 1 + i, 23)));
        int result = statement.executeUpdate();
        System.out.println(result);
    }
}
```

KdG

Slides code: preparedStatement

Exercise:

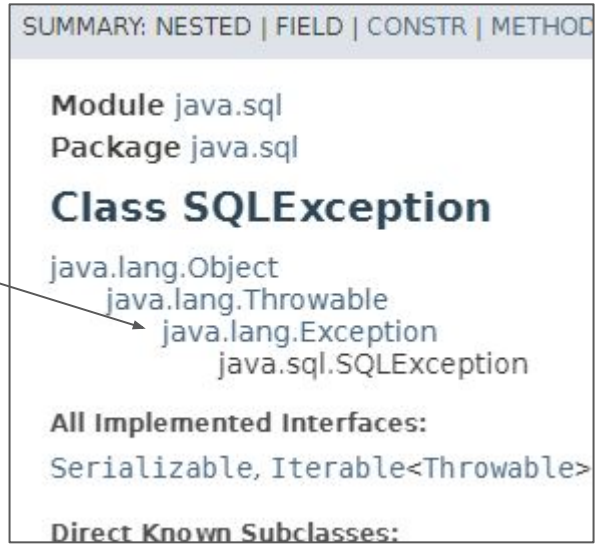
- Use a PreparedStatement in the previous exercise: does it solve the SQL Injection problem?

KdG

SQLExceptions

- If something goes wrong, JDBC throws an SQLException
  - Example: DB offline, wrong table, column does not exist, …
  - It is a *checked* exception: you have to write exception handling code for it…
  - Not very fine-grained: you get an SQLException for all kinds of problems

Java API Documentation: SQLException
inherits from Exception
so it is a **checked** exception…



KdG

Transactions in JDBC

- Sometimes you need to perform more than one query together.
- Example: transfer money to other account
  - Query one: remove money from the first account
  - Query two: add money to the second account
    - If for some reason the second query does not succeed, the first one should roll back!
- In a relational database you use transactions for this
- You can also perform transactions using JDBC

KdG

Transactions in JDBC

```
try (Connection connection = DriverManager.getConnection("jdbc:h2:mem:personsdb","sa","")) {
 connection.setAutoCommit(false);
 try (Statement statement = connection.createStatement()) {
  try {
   statement.executeUpdate("""
      INSERT
      INTO persons(name, firstname, remark)
      VALUES('Truus','Trampoline','Lastly through a hogshead of real fire!')
    """);
   if (new Random().nextBoolean()) throw new SQLException("Problem!");
   statement.executeUpdate("DELETE FROM persons WHERE firstname LIKE '%ER'");
   System.out.println("No problem, inserting and deleting...");
   connection.commit();
  } catch (SQLException e) {
   System.out.println("Problem, rolling back insert!");
   connection.rollback();
  }
}
```

We mimic an SQLException here, just for the demo. If the exception is thrown, the transaction will rollback INSERT will never be executed without DELETE!

Slides_code: persons_datasource TransactionRunner

KdG

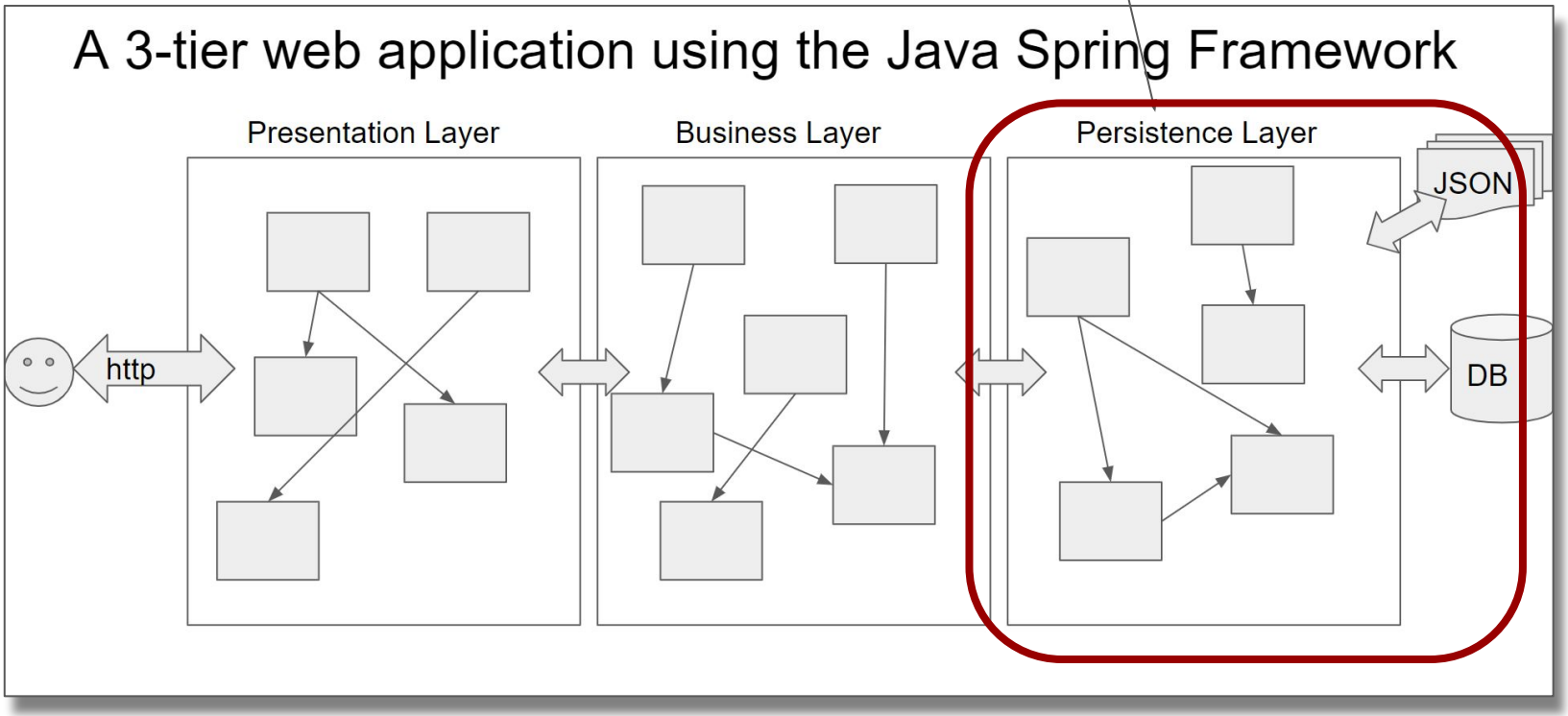
Agenda this week

JDBC
Implementing the repository
JdbcClient
Spring profiles

KdG

The persistence layer

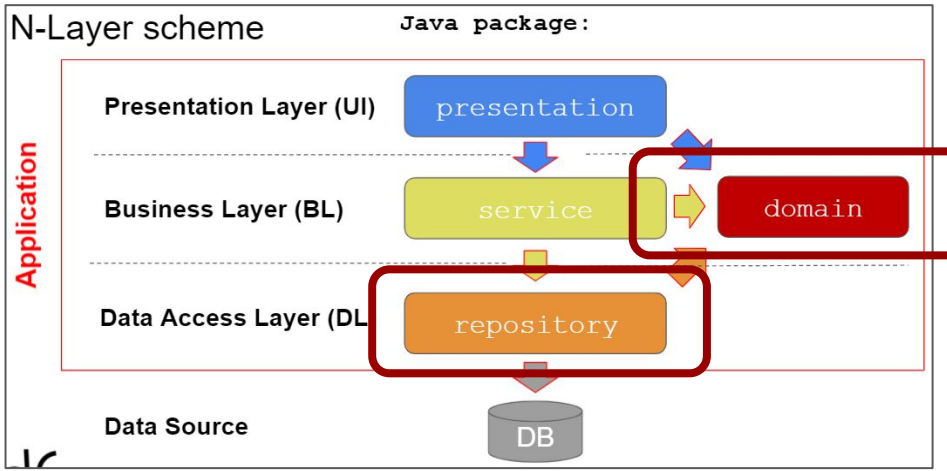
We will use JDBC in the Persistence Layer.
We will create Repository classes for it...



KdG

Remember: Domain and Repository

- Domain classes map to database tables (“entities”)
- They have an id to contain the PK of the record
- We create Repository classes for each entity.
- Annotate with @Repository (= a Spring Component)
- The Repository classes will contain the JDBC code
- Repository has methods to
  - Query entities (findBy…)
  - Create entities (create…)
  - Update entities (update…)
  - Delete entities (delete…)



KdG

Exercise: create the StudentJdbcRepository(1/6)

- Create the domain class Student (id, name, length, birthday)
- Configure your project with a database with some Students
  - Use application.properties - schema.sql - data.sql
- Create the StudentRepository interface:

```
public interface StudentRepository {
   List<Student> findAll();
   Student createStudent(Student student);
   void updateStudent(Student student);
   void deleteStudent(int id);
}
```

KdG

Exercise: create the StudentJdbcRepository(2/6)

- Create the StudentJdbcRepository(implements StudentRepository)
- Implement the findAll() method
- You need the database URL, username and password to create the connection → values from the application.properties files can be retrieved as follows:

```
public StudentJdbcRepository(@Value("${spring.datasource.url}") String dbURL,
 @Value("${spring.datasource.username}") String user,
 @Value("${spring.datasource.password}:''") String password) {
 this.dbURL = dbURL;
 this.user = user;
 this.password = password;
}
```

:'' default value
(if property is not found)
is an empty string

KdG

Exercise: create the StudentJdbcRepository(3/6)

- Create the presentation class StudentMenu
  - The StudentRepository is injected (*we will skip the service layer for this exercise…*)
  - We will show a small console menu like this:
- Test findAll!

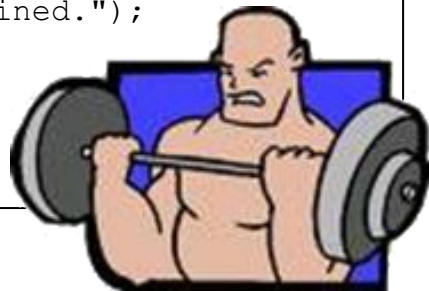
```
Welcome to the Student Management System
========================================
1) List students
2) Add student
3) Update student
4) Delete student
Make a choice:
```

KdG

Exercise: create the StudentJdbcRepository(4/6)

- Implement the createStudent() method
- You need the id of the newly created Student. How?

```
try (PreparedStatement statement
  = connection.prepareStatement("INSERT INTO students(..)", Statement.RETURN_GENERATED_KEYS)){
   //…
   int result = statement.executeUpdate();
   if (result != 0) {
       try (ResultSet generatedKeys = statement.getGeneratedKeys()) {
           if (generatedKeys.next()) {
               createdStudent.setId(generatedKeys.getInt(1));
           }
           else {
               throw new SQLException("Creating student failed, no id obtained.");
           }
       }
   }
}
```



Exercise: create the StudentJdbcRepository(5/6)

- Implement the updateStudent() and deleteStudent() methods
- Example run:

```
Welcome to the Student Management System
========================================
1) List students
2) Add student
3) Update student
4) Delete student
Make a choice:3
Student{id=1, name='jane', length=1.87, birthday=2010-06-08}
Student{id=2, name='marianne', length=1.23, birthday=1978-05-02}
Student{id=3, name='Truus', length=2.0, birthday=1954-03-12}
Student{id=4, name='An', length=1.67, birthday=1987-02-23}
Which student (id)?4
new name:Jef
new length:1.87
new birthday (mm-dd-yyyy):02-23-1987
```

KdG

Exercise: create the StudentJdbcRepository(6/6)

- What about those SQLExceptions?
- We don’t want the repository to throw SQLExceptions to the higher layers
  - Create an unchecked exception class DatabaseException
  - Wrap the SQLException in it and throw to higher layers
  - Catch in the presentation layer and show a message…

KdG

Agenda this week

JDBC
Implementing the repository
JdbcClient
Spring profiles

KdG

JdbcClient: removes the boilerplate code

```java
@Component
public class JdbcClientRunner implements CommandLineRunner {
 private JdbcClient jdbcClient;

 public JdbcClientRunner(JdbcClient jdbcClient) {
   this.jdbcClient = jdbcClient;
 }

 @Override
 public void run(String... args) throws Exception {
  //Query the database
  jdbcClient.sql("SELECT * FROM persons")
    .query((RowCallbackHandler) rs ->
       System.out.println(rs.getString("id") + " " + rs.getString("name")));
 }
}
```

Spring provides JdbcClient.
It handles connections, statement, closing, exceptions etc behind the scenes, taking parameters (url, user, password) from application.properties

RowCallbackHandler is executed for each resultset row

Old JDBC Code:

```java
public void run(String... args) throws Exception {
   try (Connection connection = DriverManager.getConnection("jdbc:h2:mem:peronsdb","sa","")){
       try (Statement statement = connection.createStatement()) {
           try (ResultSet resultSet = statement.executeQuery("SELECT * FROM students")) {
               while (resultSet.next()) {
                   System.out.println(resultSet.getString("id") + " " + resultSet.getString("name"));
               }
           }
```

KdG

Example: convert ResultSet to List of domain objects

```
public List<Person> findByName(String name){
    return jdbcClient.sql("SELECT * FROM PERSONS WHERE NAME = ?")
     .param(name)
     .query((rs, rowNum) -> new Person(rs.getInt("id"),
      rs.getString("name"),
      rs.getString("firstname"),
      rs.getString("remark")))
     .list();
}
```

A RowMapper implementation (here a lambda). It provides code to convert the ResultSet into a domain Object. JdbcClient will apply it to all records in the ResultSet!

KdG

More on JdbcClient

- `For updates use the update() method`
- Passing parameters
  - param can have an optional first parameter with the number or the name of the parameter
  - you can chain multiple param(...) calls
  - you can pass multiple parameters to the params(...) call
    - params can also take a map with name/value pairs
- `JdbcClient internally uses a PreparedSatement`

KdG

Example: query for a single object

```
public Person findById(int id) {
 return jdbcClient.sql("SELECT * FROM persons WHERE id = :personid")
   .param("personid",id)
   .query(this::mapRow)
   .single();
}

private Person mapRow(ResultSet rs, int i) throws SQLException {
 return new Person(rs.getInt("id"),
    rs.getString("name"),
    rs.getString("firstname"),
    rs.getString("remark"));
}
```

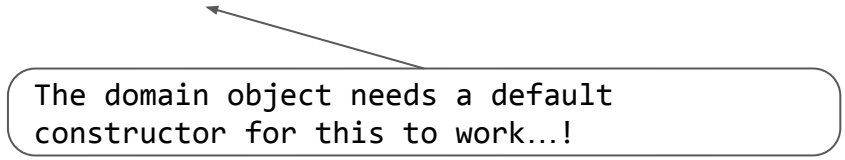
named parameter

The mapRow method reference does the same as the lambda in the previous example, but can be reused in other queries returning Person(s)

KdG

The RowMapper

- You can pass a `RowMapper` to the `query` method. It maps a row in the `ResultSet` to the Domain object. It can be done in different ways:
  - Implement a `RowMapper` (using class, a lambda expression, referencing a method from the repository…)
  - Pass the Domain class to generate the `RowMapper` based on the Domain object



Slides code: persons_datasource RowMapperRunner

KdG

Example: RowMapper from domain class

```
public List<Person> findByFirstName(String firstName) {
 return
  jdbcClient.sql("SELECT * FROM persons WHERE firstname = ?")
    .param(firstName)
    .query(Person.class)
    .list();
}
```

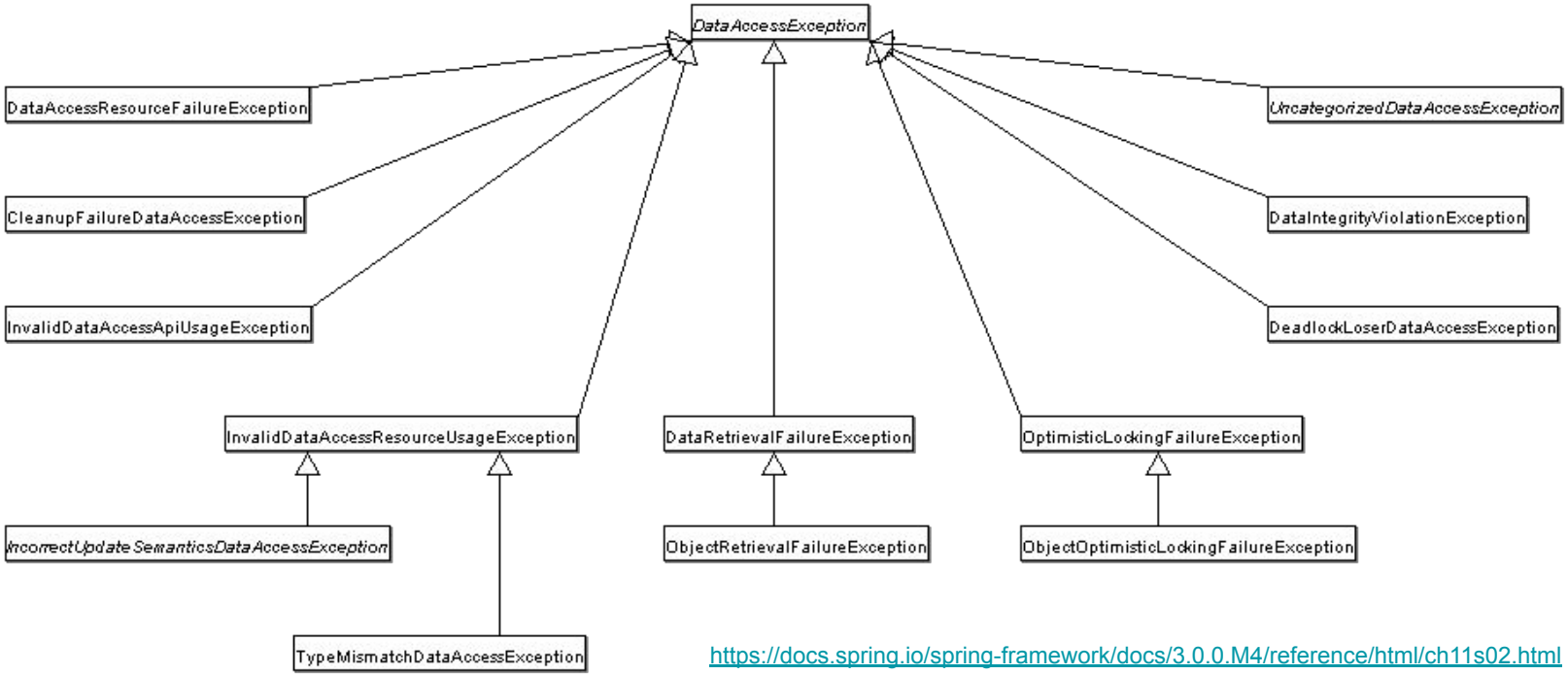
- Works only if column names match setter names

KdG

And the SQLException?

Spring provides more fine-grained exceptions so you can write better exception handling.
The exceptions are unchecked: you don't need to handle them!

- In a @Repository it is replaced by a hierarchy of `DataAccessExceptions`



https://docs.spring.io/spring-framework/docs/3.0.0.M4/reference/html/ch11s02.html

KdG

Saving entities to the database

- How is the retrieving of the id implemented?

```
public Person save(Person person) {
 JdbcClient.update("INSERT INTO persons (name, firstname, remark) VALUES (?, ?, ?)",
  person.getName(),
  person.getFirstName(),
  person.getRemark());
 // set Person id??
 return person;
}
```

KdG

Using a KeyHolder

```
public Person save(Person person) {
 KeyHolder keyholder = new GeneratedKeyHolder();
 jdbcClient.sql("""
   INSERT INTO persons(name, firstname,remark)
    VALUES (:name, :firstname,:remark)
   """)
  .params(Map.of(
      "name", person.getName(),
      "firstname", person.getFirstName(),
      "remark", person.getRemark()))
  .update(keyholder);
 person.setId(keyholder.getKey().intValue());
 return person;
}
```

KdG

Or using a SimpleJdbcInsert object

```
private SimpleJdbcInsert personInserter;
private JdbcClient jdbcClient;

public PersonJdbcClientRepository(JdbcClient jdbcClient, DataSource dataSource) {
 this.jdbcClient = JdbcClient;
 personInserter = new SimpleJdbcInsert(dataSource).withTableName("persons")
  .usingGeneratedKeyColumns("id");
}
//...
public Person save(Person person) {
 int personId = personInserter.executeAndReturnKey(Map.of(
  "name", person.getName(),
  "firstname", person.getFirstName(),
  "remark", person.getRemark())
 ).intValue();
 person.setId(personId);
 return person;
}
```

JdbcTemplate

- JdbcClient is a wrapper around Spring's JdbcTemplate
  - JdbcClient it has a simpler fluent API
  - JdbcClient is available from Spring Framework 6.1/Spring Boot 3.2
  - JdbcTemplate calls the standard JDBC API
- for complex cases/older Spring versions your may want to use the underlying JdbcTemplate
- You can find some JdbcTemplate examples in `Slides code: persons_datasource JdbcTemplateRunner`

KdG

Exercise:

- Create a second implementation of the StudentRepository, this time using JdbcClient!
  - Inject the JdbcClient
  - Use a private mapRow method to map a ResultSet entry to a Student
  - Use SimpleJdbcInsert to retrieve the id when creating a new Student
  - Think about exception handling…
- Test this implementation…

KdG

Agenda this week

JDBC
Implementing the repository
JdbcClient
Spring profiles

KdG

Spring profiles

- https://www.baeldung.com/spring-profiles
- If you like to use another DB technology for development versus testing or production?
  - Spring provides the possibility to define *profiles*
    - You can select the active profile(s) in `application.properties`:
      - `spring.profiles.active=prod`
      - `spring.profiles.active=prod,JdbcTemplate`
    - You can create separate `application-dev.properties`, `application-prod.properties`, …
  - In your code: using the `@Profile` annotation you can map beans to different profiles

KdG

Example: H2 for development - PostgreSQL for production

- Add `org.postgresql:postgresql` dependency is to build.gradle.kts
- Put PostgreSql attributes into application-prod.properties

```
spring.datasource.url=jdbc:postgresql:personsdb
spring.datasource.username=postgres
spring.datasource.password=Student_1234
spring.sql.init.mode=always
spring.sql.init.schema-locations=classpath:schema-prod.sql
spring.sql.init.data-locations=classpath:data-prod.sql
```

Normally you will not reinitialise your production db, and only use the schema and data files for development.

- PostgreSql dialect may differ

```
DROP TABLE IF EXISTS persons;
CREATE TABLE persons(
   id  SERIAL PRIMARY KEY,
   name CHARACTER VARYING(100) NOT NULL,
   firstname CHARACTER VARYING(50)  NOT NULL,
   remark CHARACTER VARYING(256)
);
```

KdG

Slides code: profiles

Or: Configure the database from code with @Profile

- Annotating beans with @Profile(“dev”) and @Profile(“prod”), will only use them if these profiles are active
- Create two @Configuration classes H2DatabaseConfig and PostgreSqlDatabaseConfig

dev

```
@Bean
public DataSource dataSource(){
   DataSource dataSource = DataSourceBuilder
.create()
        .driverClassName("org.h2.Driver")
            .url("jdbc:h2:mem:studentdb")
            .username("sa")
            .password("")
            .build();
   return dataSource;
}
```

prod

```
@Bean
public DataSource dataSource(){
   DataSource dataSource = DataSourceBuilder
.create()

.driverClassName("org.postgresql.Driver")
            .url("jdbc:postgresql:pro3_db")
            .username("postgres")
            .password("Student_1234")
            .build();
   return dataSource;
}
```

KdG

Slides code: profiles_code

Configuring @Profile from code

- Add `H2LoadData` and `PostgresLoadData` class (annotate with `@Component` and `@Profile`)
- Add `@PostConstruct` method in each class:

dev

```
@PostConstruct
public void loadData() {
 jdbcClient.sql("DROP TABLE IF EXISTS PERSONS").update();
 jdbcClient.sql("""
    CREATE TABLE PERSONS(
       ID INTEGER AUTO_INCREMENT PRIMARY KEY,
       NAME CHARACTER VARYING(100) NOT NULL,
       FIRSTNAME CHARACTER VARYING(50)  NOT NULL,
       REMARK CHARACTER VARYING(256)
    )
       """).update();
 jdbcClient.sql("""
  INSERT INTO PERSONS(NAME, FIRSTNAME,REMARK)
 VALUES ('JONES', 'JACK','of all trades'),
       ('POTTER', 'JACK','Lilly''s dad'),
       ('POTTER', 'MIA','Lilly''s mum'),
       ('REED', 'JACK','union');
       """).update();
}
```

prod

```
@PostConstruct
public void loadData() {
 jdbcClient.sql("DROP TABLE IF EXISTS PERSONS").update();
 jdbcClient.sql("""
  CREATE TABLE PERSONS(
  ID  SERIAL PRIMARY KEY,
  NAME CHARACTER VARYING(100) NOT NULL,
  FIRSTNAME CHARACTER VARYING(50)  NOT NULL,
  REMARK CHARACTER VARYING(256)
  );
  """).update();
 jdbcClient.sql("""
  INSERT
  INTO PERSONS(NAME, FIRSTNAME,REMARK)
   VALUES ('JONES', 'JACK','of all trades'),
       ('POTTER', 'JACK','Lilly''s dad'),
       ('POTTER', 'MIA','Lilly''s mum'),
       ('REED', 'JACK','union');
       """).update();
}
```

Profiles: there’s more…

- Profiles can be configured in many other ways and can be used for many other purposes, we only look at a small example
- For more info, check the tutorial: https://www.baeldung.com/spring-profiles

KdG

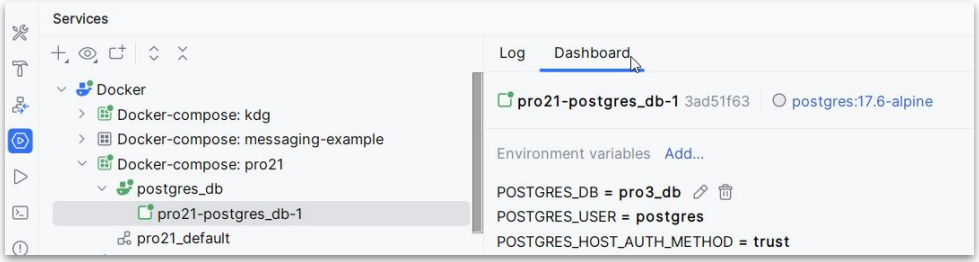
Exercise: using profiles

- Install the PostgreSQL database system on your platform
- Create a database using PostgreSQL tooling
- Try to connect using the IntelliJ Database tooling
- Walk through the previous slides and try to create 2 profiles, switch between the two and test your application.
    - Try to configure the databases using 2 application.properties files
    - Try to configure the databases from code using the @Profile annotation

KdG

Exercise: using profiles (continued)

If you are familiar with using Docker, you can add a docker-compose.yml file to your project root directory. Be sure the docker engine is running and start the postgresql database from within Intellij!
In the directory with the docker-compose.yml run
docker compose up -d



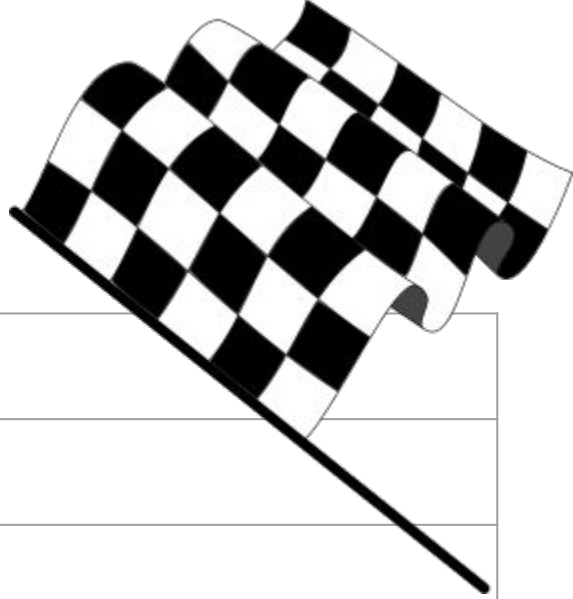
docker-compose.yml

```yaml
version: '3.8'
services:
 postgres db:
   image: postgres:17.6-alpine
   restart: always
   environment:
     POSTGRES DB: 'pro3 db'
     POSTGRES USER: 'postgres'
     POSTGRES_PASSWORD: 'Student_1234'
   ports:
     - '5444:5432'
   volumes:
      - ./postgres/data:/var/lib/postgresql/data
```

Uses port 5444 when running from docker to avoid conflicts with a possible local installation of postgres

KdG

Agenda this week



JDBC
Implementing the repository
JdbcClient
Spring profiles

KdG

Project

- Implement the Repository of your 3 entities using Spring JDBC (JdbcClients) in combination with a H2 database
- Use a schema.sql and data.sql to load initial data.
- Use profiles to be able to switch between the old implementation (using Java Collections) and the new implementation (using JDBC)

KdG